OOP Concepts & Design Patterns

Real-World Examples in Java

PART 1
OOP Pillars

PART 2
Creational Patterns

PART 3
Structural Patterns

PART 4
Behavioural Patterns

PART 5
Spring + Patterns

Encapsulation • Abstraction • Inheritance • Polymorphism • Singleton • Factory • Abstract Factory
Builder • Prototype • Adapter • Decorator • Facade • Composite • Strategy • Observer • Template Method

Every concept mapped to a real project scenario with complete Java code

Contents

Part 1 — OOP Pillars (Real-World)

- Encapsulation → Banking Account System
- Abstraction → Payment Gateway
- Inheritance → E-Commerce Product Catalogue
- Polymorphism → Notification Service

Part 2 — Creational Patterns

- Singleton → Database Connection Pool
- Factory Method → Notification Factory
- Abstract Factory→ Cross-Platform UI Toolkit
- Builder → HTTP Request Builder
- Prototype → Game Character Cloning

Part 3 — Structural Patterns

- Adapter → Third-Party Payment Adapter
- Decorator → Coffee Shop Pricing
- Facade → Home Automation System
- Composite → File System Tree

Part 4 — Behavioural Patterns

- Strategy → Discount Calculation Engine
- Observer → Stock Price Alert System
- Template Method→ Report Generator
- Command → Undo-Redo Text Editor
- Chain of Resp. → Support Ticket Escalation

Part 5 — Spring Boot Pattern Mapping

- Where Spring uses each pattern internally

Part 1 — OOP Pillars

Real-world Java implementations of the four pillars

1.1 Encapsulation — Banking Account System

Real-World Scenario: A bank must never let external code set a balance directly. All mutations must go through validated methods (deposit, withdraw). Sensitive fields like PIN and balance are hidden and accessed only through controlled public methods — exactly what Encapsulation enforces.

What problem does it solve?

- Prevents invalid states (negative balance, corrupt PIN).
- Centralises validation logic inside the class.
- Allows internal representation to change without affecting callers.

```
// ■■■ BankAccount.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class BankAccount {

    private final String accountNumber;   // immutable identifier
    private double balance;               // NEVER accessed directly from outside
    private String hashedPin;             // sensitive — never exposed raw

    public BankAccount(String accountNumber, String pin, double initialDeposit) {
        if (initialDeposit < 0) throw new IllegalArgumentException("Initial deposit cannot be negative");
        this.accountNumber = accountNumber;
        this.balance       = initialDeposit;
        this.hashedPin     = hash(pin);   // store hashed, never raw
    }

    // Controlled mutation with validation
    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Deposit must be positive");
        this.balance += amount;
        auditLog("DEPOSIT", amount);
    }

    public void withdraw(double amount, String pin) {
        verifyPin(pin);                   // authentication check
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        if (amount > balance) throw new InsufficientFundsException("Balance: " + balance);
        this.balance -= amount;
        auditLog("WITHDRAWAL", amount);
    }

    // Read-only access to balance (no setter exposed)
    public double getBalance(String pin) {
        verifyPin(pin);
        return balance;
    }

    public String getAccountNumber() { return accountNumber; } // OK to expose

    private void verifyPin(String pin) {
        if (!hash(pin).equals(hashedPin))
            throw new SecurityException("Invalid PIN");
    }
    private String hash(String pin)  { /* BCrypt or SHA-256 */ return pin + "_hashed"; }
    private void   auditLog(String op, double amt) { /* log to audit service */ }
}

// ■■■ Usage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
BankAccount acc = new BankAccount("ACC-001", "1234", 5000.0);
acc.deposit(1000.0);
acc.withdraw(500.0, "1234");
double bal = acc.getBalance("1234");   // 5500.0

// acc.balance = -99999;   // COMPILE ERROR — private field
// acc.hashedPin = "hacked"; // COMPILE ERROR — private field
```

Spring Boot parallel: @Entity classes in JPA use encapsulation. Fields are private, accessed through getters/setters. Hibernate uses reflection internally but your service layer never touches fields directly.

1.2 Abstraction — Payment Gateway

Real-World Scenario: An e-commerce platform supports Stripe, PayPal, and Razorpay. The checkout service must not care HOW each gateway processes payments — it only needs to know THAT it can charge, refund, and verify. Abstraction hides the gateway-specific complexity behind a clean interface.

```
// ■■■ PaymentGateway.java (abstraction layer) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public interface PaymentGateway {
    PaymentResult  charge(PaymentRequest request);
    RefundResult   refund(String transactionId, double amount);
    PaymentStatus  getStatus(String transactionId);
}

// ■■■ StripeGateway.java (concrete implementation) ■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Service("stripe")
public class StripeGateway implements PaymentGateway {

    private final StripeClient stripeClient;   // Stripe SDK — hidden detail

    @Override
    public PaymentResult charge(PaymentRequest req) {
        // Stripe-specific API call — caller never sees this complexity
        Charge charge = stripeClient.charges().create(
            ChargeCreateParams.builder()
                .setAmount((long)(req.amount() * 100))  // Stripe uses cents
                .setCurrency(req.currency())
                .setSource(req.token())
                .build()
        );
        return new PaymentResult(charge.getId(), charge.getStatus());
    }

    @Override
    public RefundResult refund(String txId, double amount) {
        Refund r = stripeClient.refunds().create(
            RefundCreateParams.builder().setCharge(txId).build());
        return new RefundResult(r.getId(), r.getStatus());
    }

    @Override
    public PaymentStatus getStatus(String txId) {
        Charge c = stripeClient.charges().retrieve(txId);
        return PaymentStatus.from(c.getStatus());
    }
}

// ■■■ RazorpayGateway.java (another impl, totally different API) ■■■■■■■■■■■■■■
@Service("razorpay")
public class RazorpayGateway implements PaymentGateway {
    @Override
    public PaymentResult charge(PaymentRequest req) { /* Razorpay-specific */ return null; }
    @Override
    public RefundResult refund(String txId, double amount) { return null; }
    @Override
    public PaymentStatus getStatus(String txId) { return null; }
}

// ■■■ CheckoutService.java — uses abstraction, doesn't care which gateway ■■■■
@Service
public class CheckoutService {

    private final PaymentGateway gateway;   // abstraction — not concrete class

    public CheckoutService(@Qualifier("stripe") PaymentGateway gateway) {
        this.gateway = gateway;             // swap to razorpay with zero code change
    }

    public Order checkout(Cart cart, String paymentToken) {
        PaymentRequest req = new PaymentRequest(cart.total(), "INR", paymentToken);
        PaymentResult  result = gateway.charge(req);   // abstracted call
```

```
        if (result.isSuccess()) {
            return orderService.createOrder(cart, result.transactionId());
        }
        throw new PaymentFailedException(result.errorMessage());
    }
}
```

1.3 Inheritance — E-Commerce Product Catalogue

Real-World Scenario: An e-commerce platform sells Electronics, Clothing, and Perishables. All products share common attributes (id, name, price, stock) and behaviours (applyDiscount, isAvailable). But each type has unique properties: Electronics have warranty, Clothing has size/colour, Perishables have expiry. Inheritance eliminates duplication while allowing specialisation.

```
// ■■■ Product.java (parent) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@MappedSuperclass   // JPA: fields mapped in subclass tables
public abstract class Product {

    @Id @GeneratedValue private Long id;
    private String  name;
    private double  price;
    private int     stockQuantity;
    private String  category;

    // Shared behaviour
    public boolean isAvailable()           { return stockQuantity > 0; }
    public void    reduceStock(int qty)     { this.stockQuantity -= qty; }

    // Template method — subclasses define their own discount logic
    public abstract double calculateDiscount();

    public double getFinalPrice() {
        return price - calculateDiscount();     // uses overridden method
    }

    // Common toString for all products
    @Override public String toString() {
        return String.format("[%s] %s — Rs.%.2f", category, name, getFinalPrice());
    }
}

// ■■■ Electronics.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Entity
@DiscriminatorValue("ELECTRONICS")
public class Electronics extends Product {

    private int    warrantyMonths;
    private String brand;

    @Override
    public double calculateDiscount() {
        // 15% if warranty > 12 months
        return warrantyMonths > 12 ? getPrice() * 0.15 : getPrice() * 0.05;
    }
    public boolean isUnderWarranty(LocalDate purchaseDate) {
        return purchaseDate.plusMonths(warrantyMonths).isAfter(LocalDate.now());
    }
}

// ■■■ Clothing.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Entity
@DiscriminatorValue("CLOTHING")
public class Clothing extends Product {

    private String size;
    private String colour;

    @Override
    public double calculateDiscount() {
        return getPrice() * 0.20;   // flat 20% on clothing
    }
    public boolean isAvailableInSize(String s) { return this.size.equals(s); }
}

// ■■■ Perishable.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Entity
@DiscriminatorValue("PERISHABLE")
public class Perishable extends Product {
```

```
    private LocalDate expiryDate;

    @Override
    public double calculateDiscount() {
        long daysLeft = ChronoUnit.DAYS.between(LocalDate.now(), expiryDate);
        if (daysLeft <= 2) return getPrice() * 0.50;   // 50% if expiring soon
        if (daysLeft <= 7) return getPrice() * 0.25;
        return 0;
    }
    @Override
    public boolean isAvailable() {
        return super.isAvailable() && expiryDate.isAfter(LocalDate.now());
    }
}

// ■■■ ProductService.java — polymorphic usage via parent type ■■■■■■■■■■■■■■■■
List<Product> catalogue = productRepo.findAll();
catalogue.stream()
    .filter(Product::isAvailable)
    .forEach(p -> System.out.println(p));  // each calls its own toString()
```

1.4 Polymorphism — Notification Service

Real-World Scenario: A food-delivery app must notify users via Email, SMS, or Push Notification depending on their preference. The OrderService doesn't need to know WHICH channel is used — it calls send() and runtime polymorphism dispatches to the right implementation. Adding WhatsApp later requires zero change to OrderService.

```
// ■■■ Notifier.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public interface Notifier {
    void send(String userId, String subject, String body);
    default String channel() { return "UNKNOWN"; }
}

// ■■■ EmailNotifier.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Component
public class EmailNotifier implements Notifier {
    private final JavaMailSender mailSender;
    @Override
    public void send(String userId, String subject, String body) {
        String email = userService.getEmail(userId);
        SimpleMailMessage msg = new SimpleMailMessage();
        msg.setTo(email); msg.setSubject(subject); msg.setText(body);
        mailSender.send(msg);
    }
    @Override public String channel() { return "EMAIL"; }
}

// ■■■ SmsNotifier.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Component
public class SmsNotifier implements Notifier {
    private final TwilioClient twilioClient;
    @Override
    public void send(String userId, String subject, String body) {
        String phone = userService.getPhone(userId);
        twilioClient.messages().create(phone, body);
    }
    @Override public String channel() { return "SMS"; }
}

// ■■■ PushNotifier.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Component
public class PushNotifier implements Notifier {
    private final FirebaseMessaging firebase;
    @Override
    public void send(String userId, String subject, String body) {
        String token = deviceTokenRepo.findByUserId(userId);
        Message msg = Message.builder().setToken(token)
            .setNotification(Notification.builder()
                .setTitle(subject).setBody(body).build())
            .build();
        firebase.send(msg);
    }
    @Override public String channel() { return "PUSH"; }
}

// ■■■ NotificationRouter.java — runtime polymorphism ■■■■■■■■■■■■■■■■■■■■■■■■■
@Service
public class NotificationRouter {

    private final Map<String, Notifier> notifiers;

    // Spring injects all Notifier beans into the map keyed by channel()
    public NotificationRouter(List<Notifier> allNotifiers) {
        this.notifiers = allNotifiers.stream()
            .collect(Collectors.toMap(Notifier::channel, n -> n));
    }

    public void notify(String userId, String subject, String body) {
        String pref   = userPreferenceRepo.getNotificationChannel(userId);
        Notifier n    = notifiers.getOrDefault(pref, notifiers.get("EMAIL"));
        n.send(userId, subject, body);  // runtime dispatch — polymorphism!
```

```
    }
}

// ■■■ OrderService.java — completely decoupled from channel logic ■■■■■■■■■■■■
@Service
public class OrderService {
    private final NotificationRouter router;

    public void placeOrder(Order order) {
        orderRepo.save(order);
        router.notify(order.userId(),
            "Order Confirmed #" + order.id(),
            "Your order will arrive by " + order.estimatedDelivery());
    }
}
```

Part 2 — Creational Patterns

Control how objects are created — flexibility, reuse, and resource efficiency

2.1 Singleton — Database Connection Pool

Real-World Scenario: Creating a new DB connection for every request is catastrophically expensive. A connection pool (HikariCP) must be a single shared instance across the entire application. Singleton ensures one pool exists, shared by all service threads.

```
// ■■■ DatabaseConnectionPool.java (thread-safe Singleton) ■■■■■■■■■■■■■■■■■■
public class DatabaseConnectionPool {

    // volatile ensures visibility across threads (no caching in registers)
    private static volatile DatabaseConnectionPool INSTANCE;
    private final HikariDataSource dataSource;

    private DatabaseConnectionPool() {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:postgresql://localhost:5432/shopdb");
        config.setMaximumPoolSize(20);          // max 20 concurrent connections
        config.setMinimumIdle(5);
        config.setConnectionTimeout(30_000);
        this.dataSource = new HikariDataSource(config);
        System.out.println("Connection pool initialised — ONE TIME");
    }

    // Double-checked locking — thread-safe, efficient
    public static DatabaseConnectionPool getInstance() {
        if (INSTANCE == null) {                         // first check (no lock)
            synchronized (DatabaseConnectionPool.class) {
                if (INSTANCE == null) {                 // second check (with lock)
                    INSTANCE = new DatabaseConnectionPool();
                }
            }
        }
        return INSTANCE;
    }

    public Connection getConnection() throws SQLException {
        return dataSource.getConnection();
    }

    public void shutdown() { dataSource.close(); }
}

// ■■■ Usage across different services (same instance) ■■■■■■■■■■■■■■■■■■■■■■■■
// In UserRepository:
Connection conn = DatabaseConnectionPool.getInstance().getConnection();

// In OrderRepository:
Connection conn = DatabaseConnectionPool.getInstance().getConnection(); // SAME pool

// Spring Boot equivalent (Spring manages the singleton):
@Configuration
public class DataSourceConfig {
    @Bean  // @Bean is Singleton scope by default
    public DataSource dataSource() {
        return new HikariDataSource(hikariConfig());
    }
}
```

2.2 Factory Method — Notification Factory

Real-World Scenario: A logistics app sends different types of notifications based on event type: ORDER_PLACED → Email, DELIVERY_OTP → SMS, PROMO → Push. The factory method centralises object creation, so callers never use 'new' directly. Adding a new notification type means adding a class + one factory case — no other change.

```
// ■■■ NotificationFactory.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class NotificationFactory {

    // Factory method — hides 'new', returns abstraction
    public static Notification create(NotificationEvent event) {
        return switch (event.type()) {
            case ORDER_PLACED    -> new EmailNotification(event);
            case DELIVERY_OTP    -> new SmsNotification(event);
            case PROMO_CAMPAIGN  -> new PushNotification(event);
            case FRAUD_ALERT     -> new SmsNotification(event).withHighPriority();
            default -> throw new UnsupportedEventException(event.type().name());
        };
    }
}

// ■■■ Notification.java (product interface) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public interface Notification {
    void deliver();
    String getSummary();
}

// ■■■ EmailNotification.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class EmailNotification implements Notification {
    private final NotificationEvent event;
    EmailNotification(NotificationEvent event) { this.event = event; }

    @Override
    public void deliver() {
        String html = templateEngine.render("order-placed.html", event.data());
        mailService.sendHtml(event.userId(), event.subject(), html);
        System.out.println("Email sent for: " + event.type());
    }
    @Override public String getSummary() { return "EMAIL → " + event.subject(); }
}

// ■■■ EventDispatcher.java — uses factory, never knows concrete types ■■■■■■■■■■
@Service
public class EventDispatcher {

    @EventListener
    public void onNotificationEvent(NotificationEvent event) {
        Notification notification = NotificationFactory.create(event); // factory
        notification.deliver();   // polymorphic call
        auditLog.record(notification.getSummary());
    }
}

// ■■■ Spring's own Factory usage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// BeanFactory.getBean() is a factory method
// ResponseEntity.ok(), ResponseEntity.badRequest() are factory methods
// Optional.of(), Optional.empty() are factory methods
```

2.3 Abstract Factory — Cross-Platform UI Toolkit

Real-World Scenario: A Java desktop application (JavaFX-based) must render native-looking components on Windows, macOS, and Linux. The Abstract Factory creates families of related widgets (Button, TextField, Dialog) that look native on each OS — without the application code knowing which OS it's running on.

```
// ■■■ Widget interfaces (products) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
interface Button   { void render(); void onClick(Runnable handler); }
interface TextField{ void render(); String getValue(); }
interface Dialog   { void show(String title, String message); }

// ■■■ Abstract factory ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
interface UIFactory {
    Button    createButton(String label);
    TextField createTextField(String placeholder);
    Dialog    createDialog();
}

// ■■■ Windows factory (creates Windows-native widgets) ■■■■■■■■■■■■■■■■■■■■■■■■■
class WindowsUIFactory implements UIFactory {
    @Override public Button createButton(String label) {
        return new WindowsButton(label);    // flat Win11 style
    }
    @Override public TextField createTextField(String ph) {
        return new WindowsTextField(ph);
    }
    @Override public Dialog createDialog() { return new WindowsDialog(); }
}

// ■■■ macOS factory ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class MacUIFactory implements UIFactory {
    @Override public Button createButton(String label) {
        return new MacButton(label);        // rounded Aqua style
    }
    @Override public TextField createTextField(String ph) {
        return new MacTextField(ph);
    }
    @Override public Dialog createDialog() { return new MacDialog(); }
}

// ■■■ Factory resolver — picks factory based on OS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class UIFactoryProvider {
    public static UIFactory getFactory() {
        String os = System.getProperty("os.name").toLowerCase();
        if (os.contains("win"))  return new WindowsUIFactory();
        if (os.contains("mac"))  return new MacUIFactory();
        return new LinuxUIFactory();
    }
}

// ■■■ Application.java — zero OS-specific code here ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class LoginForm {
    private final UIFactory factory;   // receives abstract factory

    public LoginForm() {
        this.factory = UIFactoryProvider.getFactory();  // resolved once
    }

    public void render() {
        TextField userField = factory.createTextField("Username");
        TextField passField = factory.createTextField("Password");
        Button    loginBtn  = factory.createButton("Sign In");

        userField.render();  // renders native to current OS
        passField.render();
        loginBtn.render();
        loginBtn.onClick(() -> authenticate(userField.getValue(), passField.getValue()));
    }
}
```

2.4 Builder — HTTP Request / Email Builder

Real-World Scenario: Building complex objects with many optional parameters is error-prone with telescoping constructors. The Builder pattern allows step-by-step construction with a fluent API. Real examples: HttpRequest.newBuilder(), StringBuilder, Lombok @Builder, UriComponentsBuilder in Spring, and email objects with optional CC, BCC, attachments.

```
// ■■■ Email.java with nested Builder ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class Email {
    // All fields final — immutable once built
    private final String   from;
    private final List<String> to;
    private final List<String> cc;
    private final String   subject;
    private final String   body;
    private final boolean  isHtml;
    private final List<Attachment> attachments;
    private final Priority priority;

    private Email(Builder b) {
        this.from        = Objects.requireNonNull(b.from, "from is required");
        this.to          = List.copyOf(b.to);
        this.cc          = List.copyOf(b.cc);
        this.subject     = Objects.requireNonNull(b.subject, "subject is required");
        this.body        = b.body;
        this.isHtml      = b.isHtml;
        this.attachments = List.copyOf(b.attachments);
        this.priority    = b.priority;
    }

    public static Builder builder() { return new Builder(); }

    // Getters only — no setters (immutable)
    public String getFrom()        { return from; }
    public List<String> getTo()    { return to; }
    public String getSubject()     { return subject; }

    // ■■■ Builder ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    public static class Builder {
        private String   from;
        private List<String> to          = new ArrayList<>();
        private List<String> cc          = new ArrayList<>();
        private String   subject;
        private String   body            = "";
        private boolean  isHtml          = false;
        private List<Attachment> attachments = new ArrayList<>();
        private Priority priority        = Priority.NORMAL;

        public Builder from(String from)        { this.from = from;           return this; }
        public Builder to(String... addrs)      { to.addAll(List.of(addrs));  return this; }
        public Builder cc(String... addrs)      { cc.addAll(List.of(addrs));  return this; }
        public Builder subject(String subject)  { this.subject = subject;     return this; }
        public Builder body(String body)        { this.body = body;           return this; }
        public Builder html(boolean isHtml)     { this.isHtml = isHtml;       return this; }
        public Builder attach(Attachment a)     { attachments.add(a);         return this; }
        public Builder priority(Priority p)     { this.priority = p;          return this; }
        public Email   build()                  { return new Email(this); }
    }
}

// ■■■ Usage — fluent, readable, no confusing parameter order ■■■■■■■■■■■■■■■■■■■■
Email email = Email.builder()
    .from("noreply@shop.com")
    .to("alice@example.com", "bob@example.com")
    .cc("manager@shop.com")
    .subject("Order #12345 Confirmed")
    .body("<h1>Your order is confirmed!</h1><p>Expected: 3 days</p>")
    .html(true)
    .priority(Priority.HIGH)
    .attach(new Attachment("invoice.pdf", invoiceBytes))
    .build();
```

```
mailService.send(email);

// ■■■ Lombok @Builder (same pattern, auto-generated) ■■■■■■■■■■■■■■■■■■■■■■■■
@Builder
@Getter
public class UserProfile {
    private final String username;
    private final String email;
    private final String avatarUrl;
    private final Instant createdAt;
}
UserProfile profile = UserProfile.builder()
    .username("alice").email("alice@example.com").build();
```

2.5 Prototype — Game Character Cloning

Real-World Scenario: In a multiplayer game, creating enemy characters from scratch for each wave is costly (loading assets, computing stats, etc.). Instead, create one 'template' character, clone it for each enemy instance, and only customise what differs. Also used in Spring's prototype bean scope and document template systems.

```
// ■■■ GameCharacter.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class GameCharacter implements Cloneable {
    private String  name;
    private int     health;
    private int     attackPower;
    private String  spriteSheet;    // expensive to load
    private List<Skill> skills;     // deep copy required!
    private Map<String, Integer> attributes;

    // Full constructor — called once for the template
    public GameCharacter(String name, int health, int attack, String sprite) {
        this.name        = name;
        this.health      = health;
        this.attackPower = attack;
        this.spriteSheet = sprite;    // reused from asset cache
        this.skills      = new ArrayList<>();
        this.attributes  = new HashMap<>();
    }

    // Prototype clone — DEEP copy (not shallow!)
    @Override
    public GameCharacter clone() {
        try {
            GameCharacter cloned = (GameCharacter) super.clone(); // shallow
            // Deep copy mutable collections
            cloned.skills     = skills.stream()
                .map(Skill::clone)
                .collect(Collectors.toList());
            cloned.attributes = new HashMap<>(this.attributes);
            return cloned;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Clone failed", e);
        }
    }

    // Fluent customisation after cloning
    public GameCharacter withName(String name)    { this.name = name; return this; }
    public GameCharacter withHealth(int hp)       { this.health = hp; return this; }
    public GameCharacter boostedAttack(int bonus) { this.attackPower += bonus; return this; }

    // Getters
    public String getName() { return name; }
    public int getHealth()  { return health; }
}

// ■■■ CharacterRegistry.java — manages prototype templates ■■■■■■■■■■■■■■■■■■■■
public class CharacterRegistry {
    private final Map<String, GameCharacter> templates = new HashMap<>();

    public void register(String key, GameCharacter template) {
        templates.put(key, template);
    }
    public GameCharacter spawn(String key) {
        GameCharacter template = templates.get(key);
        if (template == null) throw new IllegalArgumentException("Unknown character: " + key);
        return template.clone();    // clone, don't reuse
    }
}

// ■■■ Game.java — spawning enemies for a wave ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CharacterRegistry registry = new CharacterRegistry();

// One-time template creation (expensive)
GameCharacter orcTemplate = new GameCharacter("Orc", 200, 35, "orc_sprites.png");
```

```
orcTemplate.addSkill(new Skill("Berserk")); registry.register("orc", orcTemplate);

GameCharacter bossTemplate = new GameCharacter("Dragon", 5000, 150, "dragon.png");
registry.register("boss", bossTemplate);

// Fast cloning for each wave
List<GameCharacter> wave1 = IntStream.range(0, 20)
    .mapToObj(i -> registry.spawn("orc").withName("Orc-" + i))
    .collect(Collectors.toList());

GameCharacter bossFight = registry.spawn("boss").withName("Ignis the Dragon").boostedAttack(50);
```

Part 3 — Structural Patterns

Organise classes and objects into larger, flexible structures

3.1 Adapter — Third-Party Payment SDK Integration

Real-World Scenario: Your app uses a PaymentGateway interface. A new payment provider (PayU) has a completely different API signature. Instead of rewriting your service, you write an Adapter that wraps PayU's SDK and translates calls to match your interface. Java I/O uses this extensively: InputStreamReader adapts InputStream to Reader.

```
// ■■■ Your existing interface (Target) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public interface PaymentGateway {
    PaymentResult processPayment(String userId, double amount, String currency);
    boolean       refund(String transactionId);
}

// ■■■ PayU SDK (Adaptee) — cannot be changed ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Third-party library with completely different API
public class PayUSdk {
    public PayUResponse initiateTransaction(PayURequest req) { /* PayU API */ return null; }
    public boolean      cancelTransaction(String orderId, String reason) { return true; }
}
public class PayURequest {
    public String merchantKey, orderId, amount, productInfo, email;
}

// ■■■ PayUAdapter.java — bridges the two incompatible interfaces ■■■■■■■■■■■■■■■
@Component("payU")
public class PayUAdapter implements PaymentGateway {

    private final PayUSdk payuSdk = new PayUSdk();  // wrapped (adapted)

    @Override
    public PaymentResult processPayment(String userId, double amount, String currency) {
        // Translate YOUR interface call → PayU's API call
        PayURequest req = new PayURequest();
        req.merchantKey  = System.getenv("PAYU_KEY");
        req.orderId      = UUID.randomUUID().toString();
        req.amount       = String.format("%.2f", amount);  // PayU needs String!
        req.productInfo  = "Order";
        req.email        = userService.getEmail(userId);

        PayUResponse response = payuSdk.initiateTransaction(req);

        // Translate PayU response → YOUR PaymentResult
        return new PaymentResult(
            response.getMihpayid(),
            response.getStatus().equals("success") ? "SUCCESS" : "FAILED"
        );
    }

    @Override
    public boolean refund(String transactionId) {
        // PayU refund API requires different params — adapter handles translation
        return payuSdk.cancelTransaction(transactionId, "Customer requested refund");
    }
}

// ■■■ CheckoutService.java — ZERO changes needed ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Service
public class CheckoutService {
    private final PaymentGateway gateway;

    public CheckoutService(@Qualifier("payU") PaymentGateway gateway) {
        this.gateway = gateway;   // PayU via adapter — no code change here!
    }
    public void checkout(Order order) {
        PaymentResult r = gateway.processPayment(order.userId(), order.total(), "INR");
    }
}

// ■■■ Java Standard Library Adapter examples ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Arrays.asList()     adapts Array to List
// Collections.list()  adapts Enumeration to ArrayList
```

```
// InputStreamReader   adapts InputStream (bytes) to Reader (chars)
BufferedReader reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
```

3.2 Decorator — Coffee Shop Order System

Real-World Scenario: A coffee shop lets customers add extras: milk, sugar, caramel syrup, whipped cream. Each add-on changes the price and description. Instead of creating a class for every combination (LatteMilkSugar, EspressoCaramelWhip...), Decorator wraps objects dynamically. Java I/O (BufferedInputStream wrapping FileInputStream) is the canonical example.

```
// ■■■ Beverage.java (Component interface) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public interface Beverage {
    double getCost();
    String getDescription();
}

// ■■■ Base beverages (ConcreteComponents) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class Espresso implements Beverage {
    public double getCost()        { return 80.0; }
    public String getDescription() { return "Espresso"; }
}
public class Latte implements Beverage {
    public double getCost()        { return 120.0; }
    public String getDescription() { return "Latte"; }
}

// ■■■ BeverageDecorator.java (abstract Decorator) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public abstract class BeverageDecorator implements Beverage {
    protected final Beverage wrapped;
    protected BeverageDecorator(Beverage b) { this.wrapped = b; }
}

// ■■■ Concrete Decorators ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class MilkDecorator extends BeverageDecorator {
    public MilkDecorator(Beverage b) { super(b); }
    public double getCost()        { return wrapped.getCost() + 20.0; }
    public String getDescription() { return wrapped.getDescription() + ", Milk"; }
}
public class CaramelDecorator extends BeverageDecorator {
    public CaramelDecorator(Beverage b) { super(b); }
    public double getCost()        { return wrapped.getCost() + 35.0; }
    public String getDescription() { return wrapped.getDescription() + ", Caramel Syrup"; }
}
public class WhipDecorator extends BeverageDecorator {
    public WhipDecorator(Beverage b) { super(b); }
    public double getCost()        { return wrapped.getCost() + 25.0; }
    public String getDescription() { return wrapped.getDescription() + ", Whipped Cream"; }
}

// ■■■ Usage — wrap objects at runtime ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Simple Espresso
Beverage order1 = new Espresso();
System.out.println(order1.getDescription() + " = Rs." + order1.getCost());
// Espresso = Rs.80.0

// Latte + Milk + Caramel + Whip
Beverage order2 = new WhipDecorator(new CaramelDecorator(new MilkDecorator(new Latte())));
System.out.println(order2.getDescription() + " = Rs." + order2.getCost());
// Latte, Milk, Caramel Syrup, Whipped Cream = Rs.200.0

// ■■■ Spring equivalent: @Transactional uses Decorator (CGLIB Proxy) ■■■■■■■■■
// Spring wraps your @Service bean in a TransactionInterceptor decorator
// that begins a transaction before your method and commits/rolls back after.
// Your code never changes — the decorator adds behaviour transparently.
```

3.3 Facade — Home Automation System

Real-World Scenario: A smart home has complex subsystems: lighting, AC, security cameras, music, blinds. A 'Movie Mode' button should dim lights, lower blinds, set AC to 22°C, start music. The Facade pattern provides a single simplified interface (HomeController) over all the complex subsystems. Spring's JdbcTemplate is a classic Facade over JDBC.

```
// ■■■ Subsystems (complex, individual APIs) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class LightingSystem {
    void setScene(String scene) { /* call to Philips Hue API */ }
    void setBrightness(int pct) { /* 0-100 */ }
}
class AirConditioner {
    void setTemperature(int celsius) { /* MQTT command to AC unit */ }
    void setMode(String mode)        { /* COOL, HEAT, FAN */ }
}
class SecuritySystem {
    void armCameras()              { /* activate all cameras */ }
    void disarmCameras()           { /* deactivate */ }
    void lockAllDoors()            { /* smart locks API */ }
}
class MusicSystem {
    void play(String playlist)     { /* Spotify API */ }
    void setVolume(int level)      { /* 0-100 */ }
    void stop()                    { /* stop playback */ }
}
class SmartBlinds {
    void setPosition(int pct)      { /* 0=open, 100=closed */ }
}

// ■■■ HomeController.java (Facade) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Service
public class HomeController {

    private final LightingSystem   lights;
    private final AirConditioner   ac;
    private final SecuritySystem   security;
    private final MusicSystem      music;
    private final SmartBlinds      blinds;

    // ■■ One-line commands that orchestrate multiple subsystems ■■■■■■■■■■■■■■■
    public void activateMovieMode() {
        lights.setBrightness(15);          // dim lights
        lights.setScene("warm");
        blinds.setPosition(90);            // close blinds
        ac.setTemperature(22);
        ac.setMode("COOL");
        music.play("Instrumental Vibes");
        music.setVolume(30);
    }

    public void activateGoodNight() {
        lights.setScene("off");
        security.armCameras();
        security.lockAllDoors();
        music.stop();
        ac.setTemperature(24);
        blinds.setPosition(100);           // fully closed
    }

    public void activateGoodMorning() {
        blinds.setPosition(0);             // open blinds
        lights.setScene("daylight");
        lights.setBrightness(80);
        ac.setTemperature(25);
        ac.setMode("FAN");
        music.play("Morning Beats");
        music.setVolume(40);
        security.disarmCameras();
    }
}
```

```
// ■■■ Client code — trivially simple ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
homeController.activateMovieMode();   // one call, hides all complexity
homeController.activateGoodNight();

// ■■■ Spring JdbcTemplate (Facade over JDBC) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// WITHOUT facade (raw JDBC — 15+ lines):
// Connection conn = ds.getConnection();
// PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE id=?");
// ps.setLong(1, userId); ResultSet rs = ps.executeQuery();
// ... map rows ... finally { rs.close(); ps.close(); conn.close(); }

// WITH JdbcTemplate facade (1 line):
User user = jdbcTemplate.queryForObject(
    "SELECT * FROM users WHERE id=?", userRowMapper, userId);
```

3.4 Composite — File System Tree

Real-World Scenario: A file system has files and folders. A folder can contain files OR other folders. You need to calculate total size, search, and list contents — recursively. Composite lets you treat files and folders uniformly through a common interface. Also used in: UI component trees, org charts, menu hierarchies.

```
// ■■■ FileSystemComponent.java (Component) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public interface FileSystemComponent {
    String  getName();
    long    getSize();           // total size (recursive for folders)
    void    print(String indent);
    boolean search(String name);
}

// ■■■ File.java (Leaf) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class File implements FileSystemComponent {
    private final String name;
    private final long   size;
    private final String mimeType;

    File(String name, long size, String mimeType) {
        this.name = name; this.size = size; this.mimeType = mimeType;
    }
    public String  getName()           { return name; }
    public long    getSize()           { return size; }
    public boolean search(String n)    { return name.equalsIgnoreCase(n); }
    public void    print(String indent){ System.out.println(indent + "  [" + mimeType + "] " + name + " (" + size + "B)"); }
}

// ■■■ Folder.java (Composite) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class Folder implements FileSystemComponent {
    private final String name;
    private final List<FileSystemComponent> children = new ArrayList<>();

    Folder(String name) { this.name = name; }

    public void add(FileSystemComponent c)    { children.add(c); }
    public void remove(FileSystemComponent c) { children.remove(c); }

    public String getName() { return name; }

    // Recursively sums sizes of all children
    public long getSize() {
        return children.stream().mapToLong(FileSystemComponent::getSize).sum();
    }

    // Recursive search
    public boolean search(String n) {
        if (name.equalsIgnoreCase(n)) return true;
        return children.stream().anyMatch(c -> c.search(n));
    }

    // Recursive print with indentation
    public void print(String indent) {
        System.out.println(indent + "■ " + name + "/ (" + getSize() + "B total)");
        children.forEach(c -> c.print(indent + "  "));
    }
}

// ■■■ Usage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Folder root = new Folder("project");
    Folder src = new Folder("src");
        Folder main = new Folder("main");
            main.add(new File("App.java",         1200, "text/java"));
            main.add(new File("UserService.java", 3400, "text/java"));
        src.add(main);
        Folder test = new Folder("test");
            test.add(new File("UserServiceTest.java", 2100, "text/java"));
        src.add(test);
    root.add(src);
```

```
    root.add(new File("pom.xml",   5600, "text/xml"));
    root.add(new File("README.md",  800, "text/markdown"));

root.print("");                          // recursive print
System.out.println(root.getSize());     // 13100  (total all files)
System.out.println(root.search("App.java")); // true
```

Part 4 — Behavioural Patterns

Define how objects communicate and distribute responsibility

4.1 Strategy — Discount Calculation Engine

Real-World Scenario: An e-commerce platform applies different discounts: seasonal sale, loyalty member discount, bulk purchase, coupon code. The discount algorithm must be swappable at runtime without if/else chains. Strategy encapsulates each algorithm as a separate class.

```java
// ■■■ DiscountStrategy.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@FunctionalInterface
public interface DiscountStrategy {
    double apply(Cart cart);  // returns discount amount
}

// ■■■ Concrete strategies ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class SeasonalDiscount implements DiscountStrategy {
    private final double percentage;
    SeasonalDiscount(double pct) { this.percentage = pct; }
    @Override
    public double apply(Cart cart) {
        return cart.getTotal() * (percentage / 100);
    }
}

public class LoyaltyDiscount implements DiscountStrategy {
    @Override
    public double apply(Cart cart) {
        int points = cart.getUser().getLoyaltyPoints();
        if (points >= 10000) return cart.getTotal() * 0.20;
        if (points >= 5000)  return cart.getTotal() * 0.10;
        return 0;
    }
}

public class BulkDiscount implements DiscountStrategy {
    @Override
    public double apply(Cart cart) {
        if (cart.getItemCount() >= 10) return cart.getTotal() * 0.15;
        if (cart.getItemCount() >= 5)  return cart.getTotal() * 0.08;
        return 0;
    }
}

public class CouponDiscount implements DiscountStrategy {
    private final CouponRepository couponRepo;
    CouponDiscount(CouponRepository repo) { this.couponRepo = repo; }
    @Override
    public double apply(Cart cart) {
        Coupon coupon = couponRepo.findByCode(cart.getCouponCode())
            .filter(c -> !c.isExpired())
            .orElseThrow(() -> new InvalidCouponException(cart.getCouponCode()));
        return coupon.isPercentage()
            ? cart.getTotal() * coupon.getValue() / 100
            : coupon.getValue();  // flat amount
    }
}

// ■■■ CheckoutCalculator.java – uses strategy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Service
public class CheckoutCalculator {

    public double calculateFinalPrice(Cart cart) {
        // Select best strategy (or combine multiple)
        List<DiscountStrategy> strategies = resolveStrategies(cart);

        double totalDiscount = strategies.stream()
            .mapToDouble(s -> s.apply(cart))
            .max()                                  // apply best discount only
            .orElse(0);

        return Math.max(0, cart.getTotal() - totalDiscount);
    }
```

```
    private List<DiscountStrategy> resolveStrategies(Cart cart) {
        List<DiscountStrategy> list = new ArrayList<>();
        if (isSeasonalSale())           list.add(new SeasonalDiscount(15));
        if (cart.getUser() != null)     list.add(new LoyaltyDiscount());
        if (cart.getItemCount() >= 5)   list.add(new BulkDiscount());
        if (cart.getCouponCode() != null) list.add(new CouponDiscount(couponRepo));
        return list;
    }
}

// ■■■ Lambda as Strategy (Java 8) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
DiscountStrategy flatOff = cart -> 100.0;          // Rs.100 flat off
DiscountStrategy halfOff = cart -> cart.getTotal() * 0.5;
calculator.applyDiscount(cart, flatOff);
```

4.2 Observer — Stock Price Alert System

Real-World Scenario: A stock trading platform lets users set price alerts: 'notify me when INFOSYS < Rs.1400'. Multiple observers (email alerts, SMS alerts, portfolio rebalancer) must be notified whenever a stock's price changes. Observer decouples the stock from notification logic. Spring's ApplicationEventPublisher and @EventListener implement this exact pattern.

```
// ■■■ StockObserver.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public interface StockObserver {
    void onPriceChange(String symbol, double oldPrice, double newPrice);
}

// ■■■ Stock.java (Observable/Subject) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class Stock {
    private final String symbol;
    private double price;
    private final List<StockObserver> observers = new CopyOnWriteArrayList<>();

    Stock(String symbol, double initialPrice) {
        this.symbol = symbol; this.price = initialPrice;
    }

    public void addObserver(StockObserver o)    { observers.add(o); }
    public void removeObserver(StockObserver o) { observers.remove(o); }

    public void updatePrice(double newPrice) {
        double old = this.price;
        this.price = newPrice;
        if (old != newPrice) notifyObservers(old, newPrice);
    }

    private void notifyObservers(double old, double current) {
        observers.forEach(o -> o.onPriceChange(symbol, old, current));
    }
}

// ■■■ Concrete Observers ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class PriceAlertObserver implements StockObserver {
    private final Map<String, Double> alertThresholds;  // userId -> target price
    @Override
    public void onPriceChange(String symbol, double old, double newPrice) {
        alertThresholds.forEach((userId, target) -> {
            if (old > target && newPrice <= target) {   // crossed below threshold
                notificationService.sendAlert(userId,
                    symbol + " dropped to Rs." + newPrice + " (your target: Rs." + target + ")");
            }
        });
    }
}

public class PortfolioRebalancer implements StockObserver {
    @Override
    public void onPriceChange(String symbol, double old, double newPrice) {
        double changePercent = (newPrice - old) / old * 100;
        if (Math.abs(changePercent) > 5) {  // > 5% change
            rebalancingEngine.trigger(symbol, newPrice);
        }
    }
}

public class AuditLogger implements StockObserver {
    @Override
    public void onPriceChange(String symbol, double old, double newPrice) {
        auditRepo.log(symbol, old, newPrice, Instant.now());  // always log
    }
}

// ■■■ Usage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Stock infosys = new Stock("INFY", 1450.0);
infosys.addObserver(new PriceAlertObserver());
infosys.addObserver(new PortfolioRebalancer());
```

```
infosys.addObserver(new AuditLogger());

infosys.updatePrice(1380.0);  // all 3 observers notified automatically

// ■■■ Spring Boot equivalent ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Publisher (Subject):
@Service class PriceUpdateService {
    @Autowired ApplicationEventPublisher eventPublisher;
    public void update(String symbol, double price) {
        eventPublisher.publishEvent(new StockPriceEvent(symbol, price));
    }
}
// Observer:
@Component class AlertHandler {
    @EventListener
    public void handle(StockPriceEvent event) { /* send alert */ }
}
```

4.3 Template Method — Report Generator

Real-World Scenario: Generating reports (PDF, CSV, Excel) follows the same sequence: fetch data → transform → format → export. The algorithm skeleton is fixed; only the formatting step differs. Template Method defines the skeleton in a base class and lets subclasses fill in the steps.

```
// ■■■ ReportGenerator.java (Template) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public abstract class ReportGenerator {

    // Template method — FIXED algorithm skeleton, final prevents override
    public final byte[] generate(ReportRequest request) {
        List<ReportData> rawData  = fetchData(request);      // step 1
        List<ReportData> filtered = applyFilters(rawData);   // step 2
        List<ReportData> sorted   = sortData(filtered);      // step 3
        byte[] output             = formatAndExport(sorted); // step 4 — varies
        auditLog(request, output.length);                    // step 5
        return output;
    }

    // Steps subclasses MUST implement
    protected abstract byte[] formatAndExport(List<ReportData> data);

    // Steps subclasses CAN override (hooks)
    protected List<ReportData> fetchData(ReportRequest req) {
        return reportRepository.findAll(req.getFilters());  // default
    }
    protected List<ReportData> applyFilters(List<ReportData> data) {
        return data;  // default: no extra filtering
    }
    protected List<ReportData> sortData(List<ReportData> data) {
        return data.stream()
            .sorted(Comparator.comparing(ReportData::getDate))
            .collect(Collectors.toList());
    }
    private void auditLog(ReportRequest req, int size) {
        auditRepo.record(req.getUserId(), req.getType(), size, Instant.now());
    }
}

// ■■■ PdfReportGenerator.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Component("PDF")
public class PdfReportGenerator extends ReportGenerator {
    @Override
    protected byte[] formatAndExport(List<ReportData> data) {
        Document doc = new Document();
        PdfWriter.getInstance(doc, outputStream);
        doc.open();
        data.forEach(row -> doc.add(new Paragraph(row.toFormattedString())));
        doc.close();
        return outputStream.toByteArray();
    }
}

// ■■■ CsvReportGenerator.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Component("CSV")
public class CsvReportGenerator extends ReportGenerator {
    @Override
    protected byte[] formatAndExport(List<ReportData> data) {
        StringBuilder csv = new StringBuilder("Date,Amount,Category
");
        data.forEach(row -> csv.append(row.toCsvRow()).append("
"));
        return csv.toString().getBytes(StandardCharsets.UTF_8);
    }
    @Override
    protected List<ReportData> applyFilters(List<ReportData> data) {
        // CSV exports need extra deduplication
        return data.stream().distinct().collect(Collectors.toList());
    }
}
```

```
// ■■■ ReportService.java — uses template ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Service
public class ReportService {
    private final Map<String, ReportGenerator> generators;

    ReportService(List<ReportGenerator> gens) {
        this.generators = gens.stream().collect(toMap(g -> g.getClass().getSimpleName(), g -> g));
    }

    public byte[] generateReport(ReportRequest request) {
        ReportGenerator gen = generators.get(request.getFormat());  // PDF, CSV, Excel
        return gen.generate(request);  // template method called
    }
}
```

4.4 Command — Undo/Redo Text Editor

Real-World Scenario: A text editor needs Ctrl+Z (undo) and Ctrl+Y (redo). Each user action (insert, delete, format) is encapsulated as a Command object. Commands are pushed onto a history stack. Undo pops and reverses the last command. Also used in: job queues, transaction rollback, macro recording, request logging.

```java
// ■■■ Command.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public interface Command {
    void execute();
    void undo();
    String getDescription();
}

// ■■■ InsertTextCommand.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class InsertTextCommand implements Command {
    private final Document document;
    private final int      position;
    private final String   text;

    InsertTextCommand(Document doc, int pos, String text) {
        this.document = doc; this.position = pos; this.text = text;
    }
    @Override public void execute()  { document.insert(position, text); }
    @Override public void undo()     { document.delete(position, position + text.length()); }
    @Override public String getDescription() { return "Insert '" + text + "' at " + position; }
}

// ■■■ DeleteTextCommand.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class DeleteTextCommand implements Command {
    private final Document document;
    private final int start, end;
    private String  deletedText;   // saved for undo

    DeleteTextCommand(Document doc, int start, int end) {
        this.document = doc; this.start = start; this.end = end;
    }
    @Override public void execute() {
        deletedText = document.getText(start, end);  // save before deleting
        document.delete(start, end);
    }
    @Override public void undo() { document.insert(start, deletedText); }
    @Override public String getDescription() { return "Delete from " + start + " to " + end; }
}

// ■■■ EditorHistory.java (Invoker) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class EditorHistory {
    private final Deque<Command> undoStack = new ArrayDeque<>();
    private final Deque<Command> redoStack = new ArrayDeque<>();

    public void execute(Command cmd) {
        cmd.execute();
        undoStack.push(cmd);
        redoStack.clear();          // new action clears redo stack
    }

    public void undo() {
        if (undoStack.isEmpty()) return;
        Command cmd = undoStack.pop();
        cmd.undo();
        redoStack.push(cmd);
        System.out.println("Undone: " + cmd.getDescription());
    }

    public void redo() {
        if (redoStack.isEmpty()) return;
        Command cmd = redoStack.pop();
        cmd.execute();
        undoStack.push(cmd);
        System.out.println("Redone: " + cmd.getDescription());
    }
```

```
}

// ■■■ Usage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
EditorHistory history = new EditorHistory();
history.execute(new InsertTextCommand(doc, 0, "Hello"));
history.execute(new InsertTextCommand(doc, 5, " World"));
history.execute(new DeleteTextCommand(doc, 5, 11));  // delete " World"
history.undo();   // restore " World"
history.undo();   // restore to "Hello"
history.redo();   // re-apply " World
```

4.5 Chain of Responsibility — Support Ticket Escalation

Real-World Scenario: A customer support ticket flows through handlers: L1 (FAQ bot) → L2 (junior agent) → L3 (senior agent) → L4 (manager). Each handler tries to resolve it; if it can't, it passes the ticket to the next. Used in: Spring Security filter chains, Servlet filter chains, middleware pipelines, logging frameworks.

```
// ■■■ SupportHandler.java ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public abstract class SupportHandler {
    protected SupportHandler nextHandler;

    public SupportHandler setNext(SupportHandler next) {
        this.nextHandler = next;
        return next;   // fluent chaining
    }

    public abstract void handle(SupportTicket ticket);

    protected void escalate(SupportTicket ticket) {
        if (nextHandler != null) {
            nextHandler.handle(ticket);
        } else {
            System.out.println("UNRESOLVED: Ticket #" + ticket.getId() + " needs manual review");
        }
    }
}

// ■■■ FaqBotHandler.java (L1) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class FaqBotHandler extends SupportHandler {
    private final List<String> knownIssues = List.of("password reset", "track order", "cancel order");

    @Override
    public void handle(SupportTicket ticket) {
        boolean resolved = knownIssues.stream()
            .anyMatch(issue -> ticket.getDescription().toLowerCase().contains(issue));
        if (resolved) {
            ticket.resolve("L1-BOT", "Sent FAQ article automatically");
            System.out.println("L1 Bot resolved: #" + ticket.getId());
        } else {
            System.out.println("L1 Bot cannot handle, escalating...");
            escalate(ticket);
        }
    }
}

// ■■■ JuniorAgentHandler.java (L2) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class JuniorAgentHandler extends SupportHandler {
    @Override
    public void handle(SupportTicket ticket) {
        if (ticket.getPriority() == Priority.LOW || ticket.getPriority() == Priority.MEDIUM) {
            ticket.resolve("L2-AGENT", "Resolved by junior agent after investigation");
            System.out.println("L2 Agent resolved: #" + ticket.getId());
        } else {
            System.out.println("L2 Agent escalating high-priority ticket...");
            escalate(ticket);
        }
    }
}

// ■■■ SeniorAgentHandler.java (L3) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class SeniorAgentHandler extends SupportHandler {
    @Override
    public void handle(SupportTicket ticket) {
        if (ticket.getPriority() != Priority.CRITICAL) {
            ticket.resolve("L3-SENIOR", "Resolved after deep technical analysis");
            System.out.println("L3 Senior resolved: #" + ticket.getId());
        } else {
            escalate(ticket);
        }
    }
}
```

```
// ■■■ ManagerHandler.java (L4) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class ManagerHandler extends SupportHandler {
    @Override
    public void handle(SupportTicket ticket) {
        ticket.resolve("L4-MANAGER", "Manager personally resolved and issued refund");
        System.out.println("Manager resolved: #" + ticket.getId());
        notifyVip(ticket);
    }
}

// ■■■ SupportSystem.java — build and use the chain ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
SupportHandler bot     = new FaqBotHandler();
SupportHandler junior  = new JuniorAgentHandler();
SupportHandler senior  = new SeniorAgentHandler();
SupportHandler manager = new ManagerHandler();

// Wire the chain
bot.setNext(junior).setNext(senior).setNext(manager);

// Incoming tickets enter at the start of the chain
bot.handle(new SupportTicket(1, "How do I reset password?", Priority.LOW));
// → L1 Bot resolved

bot.handle(new SupportTicket(2, "Charged twice for one order", Priority.HIGH));
// → L1 Bot cannot handle → L2 escalates → L3 Senior resolved

bot.handle(new SupportTicket(3, "System-wide billing error", Priority.CRITICAL));
// → L1 → L2 → L3 → L4 Manager resolved
```

Part 5 — Spring Boot & Patterns

Where Spring uses each design pattern internally

Spring Boot is essentially a collection of design patterns. Understanding which pattern powers which Spring feature will impress any interviewer.

Pattern	Spring Usage	What it does
Singleton	Every @Bean, @Service, @Repository, @Component	*ApplicationContext maintains one instance per context*
Factory Method	BeanFactory.getBean(), ResponseEntity.ok(), Optional.of()	*Object creation delegated to factory methods*
Abstract Factory	DataSource auto-config (H2 in test, HikariCP in prod)	*Spring selects the right factory based on classpath*
Prototype	@Scope("prototype") beans	*New bean instance returned on each getBean() call*
Builder	UriComponentsBuilder, MockMvcRequestBuilders, Lombok @Builder	*Fluent API for building complex request/URI objects*
Proxy (Decorator)	@Transactional, @Cacheable, @Async, Spring Security	*CGLIB/JDK proxy wraps bean, adds cross-cutting behaviour*
Adapter	HandlerAdapter in DispatcherServlet	*Adapts @Controller, HttpRequestHandler, Servlet to uniform API*
Facade	JdbcTemplate, RedisTemplate, RestTemplate	*Hides complex API (JDBC, Redis, HTTP) behind simple methods*
Template Method	JdbcTemplate (execute), AbstractController, JpaRepository	*Skeleton algorithm in base; subclasses fill specific steps*
Observer	ApplicationEventPublisher + @EventListener	*Publish domain events; multiple listeners react independently*
Strategy	@Qualifier to choose between multiple implementations	*Inject different implementations based on context/config*
Chain of Resp.	Spring Security FilterChain, OncePerRequestFilter	*Request passes through ordered filter chain*
Front Controller	DispatcherServlet	*Single entry point routes all requests to right handler*

Pattern Selection Quick Guide

When you need to...	Use
Need only ONE instance of something?	**Singleton**
Creating objects based on a parameter?	**Factory Method**
Creating families of related objects?	**Abstract Factory**
Too many constructor parameters?	**Builder**
Expensive object, need multiple copies?	**Prototype**
Incompatible interface from third party?	**Adapter**
Add behaviour dynamically without subclass?	**Decorator**
Simplify a complex subsystem API?	**Facade**
Tree structure, treat parts and whole same?	**Composite**
Same algorithm, swappable implementations?	**Strategy**
One event, multiple independent handlers?	**Observer**
Same steps, different implementations?	**Template Method**
Encapsulate actions, support undo?	**Command**
Pass request through a processing pipeline?	**Chain of Responsibility**